

Xemis: Fair and Robust Privacy-Preserving Data Trading based on Distributed Noise Sharing

Xinxin Xing
xxxshixiannva@buaa.edu.cn
Beihang University
Beijing, China

Banghong Qin
qinbh@buaa.edu.cn
Beihang University
Beijing, China

Qianhong Wu
qianhong.wu@buaa.edu.cn
Beihang University
Beijing, China

Yizhong Liu^{*}
liuyizhong@buaa.edu.cn
Beihang University
Beijing, China

Wangjie Qiu[†]
wangjieqiu@buaa.edu.cn
Beihang University
Beijing, China

Willy Susilo
wsusilo@uow.edu.au
University of Wollongong
Wollongong, NSW, Australia

Ruonan Chen
chenruonan@xidian.edu.cn
Xidian University
Xi'an, Shaanxi, China

Jianwei Liu
liujianwei@buaa.edu.cn
Beihang University
Beijing, China

Robert H. Deng
robertdeng@smu.edu.sg
Singapore Management University
Singapore, Singapore

Abstract

Privacy-preserving data trading allows data owners to sell data to consumers through a data trading web platform, the data market, without disclosing sensitive information in raw data. It enables legitimate data transmission and aggregation, facilitating large-scale data-driven model training. However, existing differential privacy-based approaches struggle to inject precisely calibrated noise in a trustworthy manner without revealing raw data to a third party, thus making them fail in achieving strong fairness and controllable privacy simultaneously, especially when facing malicious external adversaries or a corrupted data market.

To address these issues, we present Xemis, a decentralized privacy-preserving data trading scheme, which builds upon a new multi-party computation (MPC)-based batched distributed noise sharing protocol, b-DNS. b-DNS facilitates the generation of threshold secret shares of Gaussian samples with low overhead and Byzantine robustness. Leveraging our designed game-theoretic mechanism and batched bits expansion mechanism, b-DNS achieves at least 12.7× faster and saves at least 43.6× bandwidth compared to ODO (EUROCRYPT'06) and CSU19 (CCS'19). Building atop b-DNS, Xemis further enables a decentralized, Byzantine robust data market to perturb the shares of blinded data under a controllable, precise noise level, without revealing the raw data or the perturbed data to the market. Additionally, Xemis utilizes a distributed demo

dataset sampling-based mechanism and a Byzantine fault tolerance consensus-based method to enable fair value assessment and payment-data delivery. Consequently, Xemis achieves controllable privacy and strong fairness with Web3 compatibility under malicious market nodes, while achieving at least 3.3× faster than ZLM+24 (TIFS'24) with a 64-node market. The utility of noise-perturbed data is evaluated through image classification tasks on CIFAR-10, and when adding noise with privacy budget $\epsilon = 1$ to 63% of the training data, the model maintains an accuracy of 83%.

CCS Concepts

• Security and privacy → Distributed systems security.

Keywords

Privacy-preserving data trading, distributed system, Byzantine

ACM Reference Format:

Xinxin Xing, Yizhong Liu, Ruonan Chen, Banghong Qin, Wangjie Qiu, Jianwei Liu, Qianhong Wu, Willy Susilo, and Robert H. Deng. 2026. Xemis: Fair and Robust Privacy-Preserving Data Trading based on Distributed Noise Sharing. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792247>

1 Introduction

Data trading [22, 45] facilitates the transmission of large-scale data from data owners to consumers via data markets, thereby supporting data-driven research such as deep learning and large model training. Numerous data trading web platforms, e.g., BDEX [4] and Datarade [15], serve as intermediaries for such exchanges.

Traded data often contains sensitive personal information. For example, the U.S. Healthcare Data dataset [40] on Datarade, provided by RATEPOT, includes 2.38 million fine-grained healthcare records, such as addresses and medical conditions. This motivates the study of *privacy-preserving data trading*, which enables data owners to trade data without disclosing sensitive raw records. For instance, with patient consent, a healthcare provider may apply

^{*}Corresponding author. Also with Beijing Advanced Innovation Center for Future Blockchain and Privacy Computing, Beijing, China; and State Key Laboratory of Blockchain and Data Security, Zhejiang University, Hangzhou, Zhejiang, China

[†]Also with Beijing Advanced Innovation Center for Future Blockchain and Privacy Computing, Beijing, China



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

WWW '26, Dubai, United Arab Emirates

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792247>

privacy-preserving techniques before releasing data to a pharmaceutical company for further drug development. Privacy-preserving data trading offers a practical balance between privacy protection and data value, making it a promising direction [46].

Among widely adopted privacy-preserving techniques, differential privacy (DP) [18] is particularly attractive for data trading due to its efficient noise injection mechanism. However, trading the noise-perturbed data requires more than just privacy.

Requirement ①: strong fairness. (1) *Fair privacy compensation*: Applying privacy-preserving techniques would inevitably reduce data utility. As in Table 1, prior work adopts various pricing methods [20, 23, 26, 30, 47] to determine the price for noise-perturbed data. Considering the fairness of the price, a widely adopted approach [8, 22, 22, 28, 35, 43, 47–49] calculates the noise-perturbed data price as the raw data price minus the privacy compensation. The compensation increases with the agreed privacy level and offsets the consumer's loss in data utility. (2) *Fair value assessment*: On real-world data trading web platforms, e.g., Datarade [15], the data quality is attested by dataset demos and reputations. Similarly, even with privacy-preserving techniques adopted, the consumer should also be able to evaluate whether the raw data quality matches the posted price through a low-leak preview. (3) *Fair bidirectional delivery*: The protocol must realize atomic exchange: the consumer receives the agreed data only if the owner receives the agreed payment; otherwise, neither transfer takes place.

Requirement ②: controllable privacy. (1) *Access minimization*: The raw data is accessible only to the owner. The noise-perturbed release is available solely to the owner and the paying consumer; all other entities, including the data market, learn nothing about it unless the owner or the consumer explicitly discloses it. (2) *Controllable noise injection*: the participants negotiate a concrete privacy budget, which deterministically derives the noise distribution in differential privacy. The protocol enforces efficient noise injection under that exact budget; a rational owner cannot maximize profit by applying excessive noise while falsely claiming a lower amount. **Requirement ③: robustness.** As adversarial risks intensify [42], the data market should remain functional for strong fairness and controllable privacy even under the *corruption* attack when facing a malicious Byzantine adversary [25]. Thus, reliance on a single trusted operator should be replaced with a robust decentralized architecture. Moreover, the decentralized trading process should be compatible with existing Web3 infrastructures for deployability.

As summarized in Table 1, prior efforts have pursued strong fairness or controllable privacy under diverse robustness models. Despite these advances, the following dilemma persists.

Dilemma in access minimization and controllable noise injection. Centralized differential privacy (CDP) [8, 20, 22, 30, 35, 47] requires a trusted server to perturb raw data, thereby violating access minimization by exposing the plaintext to the server. In contrast, local differential privacy (LDP) [22, 23, 26, 28, 43] allows data owners to add noise locally, which has potential in access minimization [23, 43], but contradicts controllable noise injection by enabling a rational data owner to apply higher noise than it has claimed, ultimately harming consumer interests. Schemes [48, 49] resolve this dilemma by utilizing Homomorphic Encryption (HE)-based noise-level truthfulness verification under an honest-but-curious market

model, which is fragile under malicious adversary corruption and yields pronounced ciphertext expansion and processing overheads. **Byzantine-robust solution: a distributed noise sharing component.** Controllable noise injection mandates a budget-committed protocol, while access minimization requires that the generated noise remain hidden even from Byzantine nodes, thereby precluding collusion with the consumer to recover raw data. To address these tensions, we employ a Byzantine-robust committee that jointly samples noise under a designated budget and releases only Shamir shares to the market nodes. This design enforces the budget while preventing Byzantine nodes from learning the noise. We refer to this primitive as *distributed noise sharing*. To our knowledge, ODO [17] and CSU19 [9] realize such functionality through numerous expensive Multi-Party Computation (MPC) gates per noise sample, limiting practical applicability in large-scale trading.

1.1 Main Contributions

- **b-DNS: an efficient batched distributed noise sharing.** To efficiently and distributively generate Shamir's secret shares of Gaussian noise under the Byzantine adversary, we propose a batched distributed noise sharing protocol, b-DNS. In b-DNS, the generated noise is unknown to any party and satisfies a designated distribution $\mathcal{N}(\mu, \sigma^2)$ under privacy budget ϵ . Our design first introduces a game-theoretic mechanism, which capitalizes on the divergent interests of rational owners and consumers to produce unbiased bit shares with only $O(1)$ Shamir's secret sharing-based MPC gates. Next, we propose a batched bits expansion mechanism, such that generating a Gaussian noise requires only 1 sharing of an unbiased bit. Consequently, b-DNS facilitates an efficient noise sampling process with constant MPC gates, making it scalable to both batch size and committee size, and suitable for a practical deployment in data trading web platforms.

- **Xemis: a Web3-compatible privacy-preserving data trading scheme.** Building atop b-DNS, Xemis assures malicious adversary resilience for *controllable privacy* and *strong fairness* through a decentralized privacy-preserving data trading scheme. (1) *For controllable privacy*: Xemis enables a decentralized data market to perturb the secret shares of blinded raw data under a specified privacy budget through b-DNS. This mechanism ensures controllable noise injection and fair privacy compensation, while restricting the access of noise-perturbed data strictly to the data owner and consumer, without exposing the raw data. (2) *For strong fairness*: Xemis introduces a value assessment mechanism through distributed sampling, and a Web3-compatible payment mechanism with Byzantine fault tolerance (BFT) consensus for bidirectional fair delivery.

- **Implementation and evaluation.** The evaluation results show that b-DNS saves up to 99.7% runtime and at least 97.7% bandwidth usage compared to ODO [17] and CSU19 [9]. Also, trading data in Xemis with a 64-node decentralized Web3 market saves at least 68.5% in the owner's runtime and 98.1% in the market's runtime compared to ZLM+24[48] and ZLR+23 [49], while the bandwidth usage in both schemes is at least 1.8× than in Xemis. Moreover, we evaluate the relationship between privacy level and the utility of perturbed data through the dataset CIFAR-10 with ResNet-18. Notably, the model trained under 63% of the noise-perturbed data with privacy budget $\epsilon = 1$ still maintains an accuracy of 83.06.

Table 1: Comparison of DP-based Privacy-Preserving Data Trading

Scheme	Pricing Model	Technique	Strong Fairness			Controllable Privacy		Robustness	
			Privacy Compensation	Value Assessment	Bidirectional Delivery	Access Minimization	Controllable Noise Addition	Web3 Compatible	Adversary Model
SPDTS [30]	-	CDP + Zero-knowledge proof	○	○	●	○	●	●	Malicious
DPDT [20]	Auction	CDP for collection task	-	-	●	○	○	○	Honest
MastDP [23]	Auction	LDP	○	○	○	●	○	○	Semi-honest
LWH+23 [26]	Perturbed data price	LDP + Stackelberg game	○	-	●	○	-	○	Semi-honest
[22, 35, 47]	\$data - \$ε (raw data price minus privacy compensation)	CDP for statistical results	●	○	●	○	●	○	Honest
CYF+23 [8]		CDP	●	●	●	○	●	○	Honest
HC23[22]		LDP for statistical results	●	○	●	○	○	○	Semi-honest
WSH+21 [43]		LDP + Game strategies	●	○	○	●	○	●	- (2-party)
LNZ+24 [28]		LDP + Contract	●	○	●	○	○	○	Semi-honest
[48, 49]		LDP + HE + Truthfulness verify	●	○	●	○	●	○	Semi-honest
Xemis		Distributed noise sharing + MPC + Consensus	●	●	●	●	●	●	Malicious

○ Not Support ● Support

2 Preliminaries

2.1 Notation

Let n denote the number of nodes in the data market, t be the threshold and maximum tolerance for malicious nodes, and $\mathbb{Z}_p$ be a field of large prime order p . We use $\mathcal{P}_o, \mathcal{P}_c$ to represent the data owner and consumer, respectively. We also use $\mathcal{P}_i$ to represent the i -th node in the data market committee. The bold symbol such as $\mathbf{a}$ denotes a matrix or a vector, $\mathbf{a}[j]$ represents the j -th entry in vector $\mathbf{a}$, while $\mathbf{a}[j, l]$ represents the (j, l) -th entry in $\mathbf{a}$. The element-wise (n, t) -secret sharing of $\mathbf{a}$ is written as $\langle \mathbf{a} \rangle$, where $\langle \mathbf{a} \rangle_i$ denote the share for the i -th node. In the distributed noise sharing protocol, $\mathcal{N}(\mu, \sigma^2)$ represents the Gaussian distribution with mean μ and variance σ^2 , and κ represents the distribution parameter. In the data trading protocol, ϵ represents the privacy budget, D represents the data scale. Moreover, $\mathbf{m}$ denotes the raw data, $\mathbf{b}$ denotes the blinding factor, and $\mathbf{c}$ denotes the blinded raw data. When introducing the Gaussian noise vector ξ , we use $\tilde{\mathbf{m}}$ to represent the noise-perturbed data, and use $\tilde{\mathbf{c}}$ to represent the perturbed and blinded data. In constructing the demo dataset, we use S to denote the sample percentage, use $\tilde{\mathbf{m}}$ to denote the sampled raw data, use $\tilde{\mathbf{b}}$ to denote the sampled blinding factor, and use $\tilde{\mathbf{c}}$ to represent the sampled blinded data. Moreover, $r, \mathbf{p}, \tilde{\mathbf{p}}$ respectively represent the Merkle root, the Merkle proof of blinding factor $\mathbf{b}$, and the Merkle proof of sampled blinding factor $\tilde{\mathbf{b}}$.

2.2 Building Blocks

Shamir's secret sharing-based MPC. Shamir's secret sharing is widely used to construct efficient and robust multi-party computation systems [3, 11], which allows mutually distrustful participants $\mathcal{P}_1, \dots, \mathcal{P}_n$ to collaboratively evaluate an arbitrary function $f(x_1, \dots, x_n)$ over their respect private inputs $x_1, \dots, x_n$. In this work, the following subprotocols are utilized:

- $\Pi_{ss}.setup(1^\lambda, n) \rightarrow (\mathbb{Z}_p, \{\langle x \rangle, \langle y \rangle, \langle z \rangle | \langle z \rangle = \langle xy \rangle\})$: Parameterized by the security parameter λ , initialize a finite field $\mathbb{Z}_p$. The multiplication gate is instantiated using scheme [5] with Beaver triples $\langle x \rangle, \langle y \rangle, \langle z \rangle$ generated in the setup phase, where $\langle z \rangle = \langle x \cdot y \rangle$.
- $\Pi_{ss}.share(\mathbf{s}) \rightarrow \{\langle \mathbf{s} \rangle_i\}_{i \in [n]}$: This protocol allows a dealer $\mathcal{P}$ to share a batch of b secrets $\mathbf{s} \in \mathbb{Z}_p^b$ among nodes $\mathcal{P}_1, \dots, \mathcal{P}_n$, such that each node $\mathcal{P}_i$ receives a share $\langle \mathbf{s} \rangle_i$. Specifically, we adopt the lightweight secret sharing schemes [39, 44] with completeness, which ensures that if an honest node outputs its share, there exists

b polynomials $f(x)$ of t -degree, such that every honest node $\mathcal{P}_i$ outputs $\langle \mathbf{s} \rangle_i = f(i)$, and if the dealer is honest, $f(0) = \mathbf{s}$.

- $\Pi_{ss}.rec(\{\langle \mathbf{s} \rangle_i\}_{i \in [n]}) \rightarrow \mathbf{s}$: This protocol performs the robust polynomial interpolation [6]. Given $n = 3t + 1$ evaluations of an unknown t -degree polynomial, of which at most t may be withheld or falsely sent from the adversary, this protocol uniquely reconstructs the secrets $\mathbf{s}$ without polynomial commitments [24].
- $\Pi_{ss}.add(\langle \mathbf{a} \rangle, \langle \mathbf{b} \rangle) \rightarrow \langle \mathbf{a} + \mathbf{b} \rangle$: Denote $\langle \mathbf{a} \rangle, \langle \mathbf{b} \rangle$ as the shares of secrets $\mathbf{a}, \mathbf{b} \in \mathbb{Z}_p$. Due to their additively homomorphic property, addition can be performed locally through $\langle \mathbf{a} + \mathbf{b} \rangle = \langle \mathbf{a} \rangle + \langle \mathbf{b} \rangle$. Note that we also use $+$ to denote $\Pi_{ss}.add$ for simplicity.
- $\Pi_{ss}.mul(\langle \mathbf{a} \rangle, \langle \mathbf{b} \rangle) \rightarrow \langle \mathbf{a} \cdot \mathbf{b} \rangle$: Given the Beaver triples generated offline and secret shares $\langle \mathbf{a} \rangle, \langle \mathbf{b} \rangle$, all nodes collectively reconstruct $(\mathbf{a} - x)$ and $(\mathbf{b} - y)$ through $\Pi_{ss}.add$ and $\Pi_{ss}.rec$, it holds that:

$$\langle \mathbf{a} \cdot \mathbf{b} \rangle = (\mathbf{a} - x)(\mathbf{b} - y) + [z] + (\mathbf{a} - x)[y] + (\mathbf{b} - y)[x]$$

Byzantine fault tolerance consensus. Byzantine fault tolerance consensus Π_{BFT} [31] allows participating nodes to agree on a totally ordered sequence of proposals. We adopt the asynchronous instantiation [12] that is setup-free with lightweight computation.

Coin-Tossing. Coin-tossing protocol Π_{coin} [37] enables a group of participants to collaboratively generate a sequence of uniform random bits. Such protocols are widely used in distributed protocols such as consensus algorithms [16, 33]. An (n, t) - Π_{coin} protocol ensures that if up to t -out-of- n participants are corrupted, the output bits remain unbiased and are agreed upon by all honest nodes.

Differential Privacy. DP is a rigorous method for safeguarding data privacy by introducing noise. For privacy budget $\epsilon \geq 0$ and $0 \leq \delta \leq 1$, a random mechanism $\mathcal{M}$ satisfies (ϵ, δ) -DP if, for any neighboring datasets X and X' that differ on at most one record, and any subset Q of the outputs of $\mathcal{M}$: $\Pr[\mathcal{M}(X) \in Q] \leq e^\epsilon \Pr[\mathcal{M}(X') \in Q] + \delta$. Gaussian mechanism (GM) [19] is a widely used differential privacy mechanism. By adding noise drawn from a Gaussian distribution $\mathcal{N}(0, \sigma^2)$, it ensures that the mechanism satisfies (ϵ, δ) -DP. The variance of the Gaussian distribution is calculated as: $\sigma^2 = 2 \ln(1.25/\delta) \Delta f^2 / \epsilon^2$, where Δf is the sensitivity.

3 Problem Formulation

3.1 Data Pricing Model

Quantifying the value of noise-perturbed data is challenging, and numerous prior work [29, 36] have studied this problem. We adopt

a classical and widely adopted [48, 49] pricing model [27] for the perturbed data: $\$_{\text{data}} = \$_{\text{raw}} - \$_{\epsilon}$, which equals the raw data price subtracting the privacy compensation. In differential privacy, a smaller privacy budget ϵ leads to increased noise, which reduces the data utility and necessitates higher privacy compensation for the data consumer, thus lowering the price of the noise-perturbed data. Conversely, larger ϵ results in less noise, which decreases privacy compensation and increases the price of the noise-perturbed data.

- **Raw data price $\$_{\text{raw}}$:** Practically, we assume that the data owner posts the raw data price $\$_{\text{raw}}$, and the data consumer can evaluate whether the price is reasonable by accessing a demo of the raw data, i.e., a randomly sampled data pack. This mechanism is common in real-world data markets, as exemplified by the web platforms, AWS Data Exchange [2] and Datarade [15].

- **Privacy compensation $\$_{\epsilon}$:** The data consumer is compensated as the trading process decreases the raw data utility by perturbing its sensitive information through differential privacy. $\$_{\epsilon}$ is calculated from the privacy budget ϵ , which determines the noise level, i.e., the standard deviation in the Gaussian mechanism.

3.2 System Model

Data Owner $\mathcal{P}_o$. The data owner possesses raw data vector $\mathbf{m}$ with scale $D = |\mathbf{m}|$ and seeks to sell its noise-perturbed version $\tilde{\mathbf{m}} = \mathbf{m} + \xi$ without leaking $\mathbf{m}$ to any other party, where ξ represents the noise vector generated under a specified privacy budget ϵ .

Data Consumer $\mathcal{P}_c$. The data consumer purchases the noise-perturbed data $\tilde{\mathbf{m}}$ at a price $\$_{\text{data}} = \$_{\text{raw}} - \$_{\epsilon}$ for training tasks. For $\mathcal{P}_c$, the value of the raw data should be assessed before payment, and the promised privacy budget should be guaranteed.

Data Market. The data market consists of n mutually distrustful nodes, operating in a decentralized manner and using BFT [31] to reach consensus on critical information, such as a refund. The market is also responsible for controllable privacy and strong fairness under the Byzantine adversary. The market nodes, owners, and consumers are disjoint sets of entities with no role overlap.

3.3 Adversary Model and Threat Model

Assume that there exists a Probabilistic Polynomial Time (P.P.T) malicious Byzantine adversary $\mathcal{A}$, capable of statically corrupting up to $t < n/3$ market nodes. A corrupted node is fully controlled by $\mathcal{A}$, who can read all its inputs and determine the outgoing messages.

Both the data owner and data consumer are *rational* entities. Any actions of the *rational* entities are driven by self-interest, including potential collusion with the corrupted market nodes. The owner $\mathcal{P}_o$ may maximize profit by (1) receiving payment from the consumer without delivering data, and (2) claiming a lower noise addition with privacy budget ϵ while applying a higher actual noise with privacy budget ϵ_{real} , satisfying $\epsilon > \epsilon_{\text{real}}$, thus getting the higher selling price, but undermining the data utility. The consumer $\mathcal{P}_c$ may gain advantages by: (1) acquiring the noise-perturbed data $\tilde{\mathbf{m}}$ without payment, and (2) obtaining $\tilde{\mathbf{m}}$ with a lower noise level under privacy budget ϵ_{real} while getting more compensation for a higher noise level under the privacy budget ϵ , where $\epsilon_{\text{real}} > \epsilon$.

3.4 Network Model

A *secure and authenticated* point-to-point (p2p) channel is employed between any two entities that are pair-wise connected, indicating

that the transmitted message is signed and encrypted. Additionally, we assume that two broadcast channels are established: one between the owner and the market nodes, and the other between the consumer and the market nodes. The broadcast channels are implemented through reliable broadcast [14] over the p2p channels, which guarantees that all nodes receive the same message broadcast by a dealer with Byzantine robustness. We also consider an asynchronous network with further details given in Appendix D.

3.5 System Goal

Suppose the price of the noise-perturbed data is $\$_{\text{data}} = \$_{\text{raw}} - \$_{\epsilon}$, $\mathbf{m}$ is the raw data of $\mathcal{P}_o$. The security goals of Xemis are defined as:

(G1) Strong fairness. (1) Fair privacy compensation: the privacy compensation $\$_{\epsilon}$ matches the noise level of the perturbed data. (2) Fair value assessment: $\mathcal{P}_c$ can evaluate whether the raw data meets its expectation through a randomly sampled demo, considering the raw data price is $\$_{\text{raw}}$. (3) Fair bidirectional delivery: either both of the following events succeed, or neither does: $\mathcal{P}_c$ obtains the noise-perturbed data $\tilde{\mathbf{m}} = \mathbf{m} + \xi$, and $\mathcal{P}_o$ receives the deposit $\$_{\text{data}}$. **(G2) Controllable privacy.** (1) Access minimization: $\mathbf{m}$ is accessible only to $\mathcal{P}_o$. The noise-perturbed data $\tilde{\mathbf{m}}$ is accessible solely to $\mathcal{P}_o$ and the paying $\mathcal{P}_c$; all other entities, including the data market, learn nothing about it unless the owner or the consumer explicitly discloses it. (2) Controllable noise injection: if $\mathcal{P}_c$ obtains the noise-perturbed data $\tilde{\mathbf{m}}$, then there must be $\tilde{\mathbf{m}} = \mathbf{m} + \xi$, where ξ is the noise vector generated from the pre-negotiated privacy budget. **(G3) Byzantine robustness.** In a decentralized committee composed of n nodes, (G1) and (G2) are satisfied with Web3 compatibility if at most $t = \lfloor 1/3n \rfloor$ nodes are corrupted by a Byzantine adversary.

4 b-DNS: Batched Distributed Noise Sharing

4.1 Scheme Overview

Figure 1 illustrates the workflow of b-DNS. Intuitively, b-DNS generates numerous (n, t) -secret shares of uniformly random bits in the integer field $\mathbb{Z}_p$, which are then accumulated and scaled to yield shares corresponding to an approximate Gaussian distribution. The public input of b-DNS includes data scale D , field $\mathbb{Z}_p$ of a large prime order p , the number of nodes n , threshold t , pre-negotiated Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$ and distribution parameter κ that controls the similarity between the generated distribution and $\mathcal{N}(\mu, \sigma^2)$. It outputs the Shamir shares of D Gaussian noise to each node.

Initially, distributed market nodes collaboratively execute coin-tossing to produce a pool of unbiased coins during the offline phase (Step ①). The data owner initiates the online phase by secret-sharing some randomly sampled bits in $\mathbb{Z}_p$ among market nodes. The market nodes then validate these bit shares through MPC gates (Step ②). Next, the data consumer reliably broadcasts some randomly sampled bits. The market nodes then conduct a random flip of the owner's shares through these bits, yielding the shares of unbiased bits (Step ③). From the resulting shares, each market node locally performs more flips through the offline-generated coin pool, thereby expanding the scale of bit shares for Gaussian noise generation (Step ④). Finally, the nodes accumulate these bit shares and scale the resulting value to obtain the shares of Gaussian samples with the designated distribution $\mathcal{N}(\mu, \sigma^2)$ (Step ⑤).

4.2 Concrete Scheme of Π_{b-DNS}

In this section, we describe the concrete scheme of Π_{b-DNS} , with the formal definition and security analysis given in the Appendix A

Our design of the distributed noise sharing protocol is inspired by ODO [17], which obtains the Shamir sharing of a Gaussian sample through $O(\kappa)$ MPC gates, where κ is the distribution parameter. In the data trading context, we make two key observations that facilitate the generation of Gaussian shares at a much lower cost:

- *Conflict of interest between the rational owner and consumer:* increased noise benefits the data owner while decreased noise favors the consumer. Intuitively, by modeling their interaction as a two-player, game-theoretic mechanism, we obtain an (n, t) -secret sharing of an unbiased bit in $\mathbb{Z}_p$: the owner and the consumer independently sample bits $b_o, b_c \in \{0, 1\}$, where b_o is shared to the participating nodes through $\Pi_{ss}.share$ and validated by MPC gates, and b_c is explicitly revealed after this. The participating nodes then securely compute $\langle b \rangle = \langle b_o \oplus b_c \rangle$ using a linear operation. As the owner expects $b = 1$ while the consumer expects $b = 0$, we prove in Appendix A.2 that the unique Nash equilibrium leading to an unbiased b in $\{0, 1\}$. Notably, this process requires only $O(1)$ constant-round, secret sharing-based MPC.

“1 sharing of unbiased bit $\Leftrightarrow O(1)$ secret sharing-based MPC”

- *Batched expansion mechanism:* assuming a data scale of D and a distribution parameter κ , the advantage of an external adversary in guessing the Gaussian sampling in ODO [17] is $2^{-\kappa D}$, as generating each sample requires sharing κ bits of randomness. However, data trading typically involves large volumes of data with a huge D , an advantage of 2^{-D} is small enough to be neglected. Thus, we devise a batched share expansion mechanism, where the market nodes utilize $\kappa \times D$ offline-generated public coin to flip the sharing of D bits, yielding the sharing of D Gaussian noise.

“1 data entry $\Leftrightarrow 1$ noise $\Leftrightarrow 1$ sharing of unbiased bit”

Step 0: offline unbiased coin generation. The goal of this phase is to publicly generate a pool of unbiased bits within the committee. This process is commonly used in distributed noise sharing [9, 17] or distributed sampling [9, 41]. For a data vector of scale D and distribution parameter κ , a vector of $D \times \kappa$ uniform random bits is generated by the market nodes collectively by invoking coin-tossing Π_{coin} . Denote **coin** as the $D \times \kappa$ matrix of these coins.

Step 1: owner's bits sharing ($\mathbb{Z}_p$). The owner samples a bit vector $\mathbf{s}$ of length D and distributes it among the market nodes through $\Pi_{ss}.share$ in the field $\mathbb{Z}_p$. Holding the sharing $\langle \mathbf{s} \rangle$, the market nodes are responsible for verifying whether each entry in $\mathbf{s}$ falls within $\{0, 1\}$, where they invoke a multiplication gate $\Pi_{ss}.mul$ and an addition gate $\Pi_{ss}.add$ to obtain the sharing $\langle \mathbf{s}^2 - \mathbf{s} \rangle$, then reconstruct through $\Pi_{ss}.rec$ for $\mathbf{s}^2 - \mathbf{s}$. If $\mathbf{s}^2 - \mathbf{s} = 0$ holds, the sharing $\langle \mathbf{s} \rangle$ is valid with $\mathbf{s} \in \{0, 1\}^D$. Each node then outputs the checkpoint $(\mathcal{P}_o, done)$ to confirm the valid behavior of the data owner $\mathcal{P}_o$.

Step 2: unbiased seeds flipping ($\mathbb{Z}_p$). When receiving at least $2t + 1$ checkpoints $(\mathcal{P}_o, done)$ from the market nodes, the data consumer randomly samples a bit vector $\mathbf{s}'$ of length D and reliably broadcasts it among market nodes through the Byzantine-resilient broadcast channel, as discussed in Section 3.4. Each market node

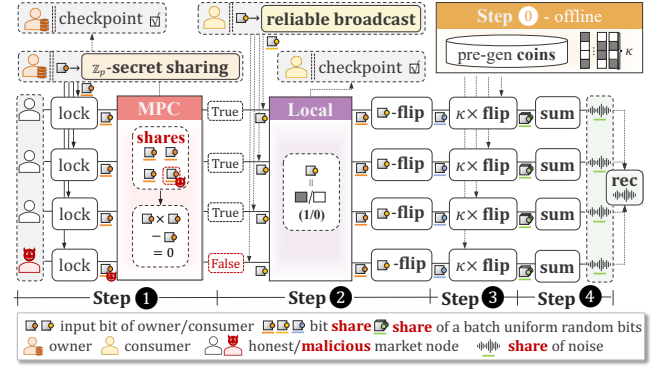


Figure 1: Workflow of batched distributed noise sharing

flips the sharing of owner's bits $\langle \mathbf{s} \rangle$ according to each entry in $\mathbf{s}'$, thus obtaining the vector $\langle \mathbf{seed} \rangle$ of unbiased bits shares:

$$\text{for } j \in [1, D] : \langle \mathbf{seed}[j] \rangle := \begin{cases} \langle \mathbf{s}[j] \rangle, & \text{if } \mathbf{s}'[j] = 0, \\ 1 - \langle \mathbf{s}[j] \rangle, & \text{else if } \mathbf{s}'[j] = 1. \end{cases}$$

This process effectively executes an element-wise XOR operation between the sharing vector $\langle \mathbf{s} \rangle$ and bit vector $\mathbf{s}'$.

Step 3: batched shares expansion ($\mathbb{Z}_p$). This phase aims to expand the shares of D -bit unbiased seeds into the shares of $D \times \kappa$ unbiased bits. Specifically, each node performs a local transformation of the unbiased seed shares $\langle \mathbf{seed} \rangle$ based on the offline-generated matrix **coin**, to produce the shares of a $D \times \kappa$ matrix of unbiased bits, denoted by $\langle \mathbf{u} \rangle$, through the following procedure:

for $j \in [1, D]$ and $l \in [1, \kappa]$:

$$\langle \mathbf{u}[j, l] \rangle := \begin{cases} \langle \mathbf{seed}[j] \rangle, & \text{if } \mathbf{coin}[j, l] = 0, \\ 1 - \langle \mathbf{seed}[j] \rangle, & \text{else if } \mathbf{coin}[j, l] = 1. \end{cases}$$

Specifically, if the element at position (j, l) in the matrix **coin** is 0, node $\mathcal{P}_i$ assigns the j -th entry of $\langle \mathbf{seed} \rangle$ directly to the element at position (j, l) in its local matrix $\langle \mathbf{u} \rangle$. Conversely, if the element is 1, node $\mathcal{P}_i$ assigns $1 - \langle \mathbf{seed}[j] \rangle$ to $\langle \mathbf{u}[j, l] \rangle$. This is equivalent to an XOR operation. Consequently, each node produces a matrix of unbiased bits shares of size $D \times \kappa$.

Step 4: gaussian noise derivation. In this phase, each node derives the (n, t) -secret share of a Gaussian noise vector with a designated distribution $\mathcal{N}(\mu, \sigma)$. Each node $\mathcal{P}_i$ first extracts each column from the matrix $\langle \mathbf{u} \rangle$ and sums all the κ columns through $\Pi_{ss}.add$. This results in a vector of Gaussian noise shares $\langle \xi \rangle$ with size D and distribution $\mathcal{N}(\kappa/2, \kappa/4)$. The noise shares with the target distribution $\mathcal{N}(\mu, \sigma)$ are then centered and scaled through:

$$\langle \xi \rangle := 2\sigma/\sqrt{\kappa}(\langle \xi \rangle - \kappa/2) + \mu$$

5 Xemis: Privacy-Preserving Data Trading

5.1 Scheme Overview

Figure 2 depicts the workflow of Xemis, which proceeds in the following six phases. Initially, the building blocks and all necessary parameters are set up (Phase 0, system setup). Next, the data owner registers the data information, blinds the raw data with a one-time pad, and distributes the blinded data to all market nodes through secret sharing (Phase 1, data registration). Market nodes

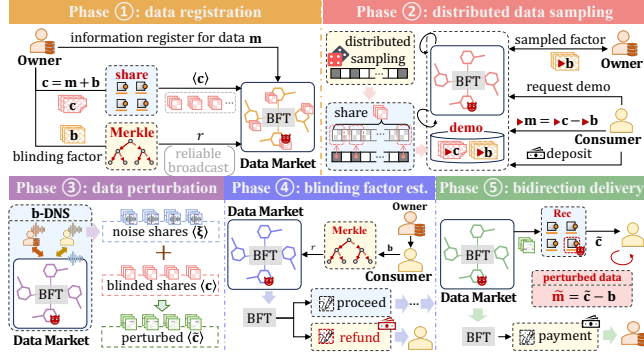


Figure 2: Workflow of Xemis

distributively construct the shares of a noise-free demo dataset, from which the consumer reconstructs to decide whether to proceed with payment (Phase ②, distributed data sampling). Upon payment, the batched distributed noise sharing protocol Π_{b-DNS} is executed, where each node receives the (n, t) -Shamir's shares of Gaussian samples. These samples are then used to perturb the shares of blinded data (Phase ③, distributed data perturbation). If the above process succeeds, the blinding factor is confidentially and verifiably established between the owner and the consumer (Phase ④, blinding factor establishment). Finally, the shares of noise-perturbed and blinded data are transferred to the data consumer, and the deposit is transferred to the data owner. After receiving enough shares, the data consumer de-blinds their reconstruction result, thus obtaining the noise-perturbed data (Phase ⑤, bidirectional fair delivery).

In the following, we first inject noise into the whole dataset, and then extend our discussion to perturb only the sensitive data portion. The security analysis is given in the Appendix B.

5.2 System Setup

Initialize n mutually distrustful nodes to set up the market (further details are given in Appendix F), with t being the fault-tolerance threshold. Given parameter λ , execute $\Pi_{ss}.setup(1^\lambda, n)$ for the Beaver triples used in $\Pi_{ss}.mul$ and field $\mathbb{Z}_p$. The committee prepares a coin pool through Step 0 in Π_{b-DNS} , and then initializes a pricing system with privacy compensation $\$_\epsilon$ calculated from ϵ .

5.3 Data Registration

In this phase, the raw data is blinded and then distributed with (n, t) -secret sharing among market nodes, while data price, privacy budget, and demo size are registered by the data owner.

- The data owner $\mathcal{P}_o$ first blinds the raw data $\mathbf{m}$ by generating a random blinding factor $\mathbf{b}$ congruent in scale with $\mathbf{m}$, and calculating the blinded data $\mathbf{c} := \mathbf{m} + \mathbf{b}$. The data owner then invoke $\Pi_{ss}.share(\mathbf{c})$ to share the blinded data among the market nodes, such that each market node obtain an (n, t) -threshold share $\langle \mathbf{c} \rangle$.
- A vector commitment r and the witnesses $\mathbf{p}$ for blinding factor $\mathbf{b}$ are calculated via the Merkle tree. The privacy budget ϵ , the raw data price $\$_{raw}$, the Merkle root r , and the demo size S are broadcast among market nodes for registration.

5.4 Distributed Data Sampling

In this phase, the data market randomly samples a demo dataset, enabling consumers to evaluate whether it is worth the price $\$_{raw}$.

- According to the demo size S and data scale D , market nodes distributively generates S uniformly random values from $\{1, \dots, D\}$, through *distributed random beacon* [7, 10, 13]. Given these S values, each node encodes them into a D -bit binary vector $\mathbf{flag}$, where $\mathbf{flag}[x] = 1$ only for every sampled value x . Note that the cost of *distributed random beacon* can be eliminated through Fiat-Shamir [44].
- Each node calculates the inner-product between $\mathbf{flag}$ and the share of blinded data for the share of blinded demo $\langle \tilde{\mathbf{c}} \rangle := \mathbf{flag} \cdot \langle \mathbf{c} \rangle$.
- Each node sends $\mathbf{flag}$ to the data owner $\mathcal{P}_o$, who calculates the sampled blinding factor $\tilde{\mathbf{b}} := \mathbf{flag} \cdot \mathbf{b}$ and the corresponding Merkle proof vector $\tilde{\mathbf{p}}$ upon receiving $\mathbf{flag}$ from $(2t + 1)$ different nodes.
- $\mathcal{P}_o$ broadcast $(\tilde{\mathbf{b}}, \tilde{\mathbf{p}})$ among market nodes. Each node validates $(\tilde{\mathbf{b}}, \tilde{\mathbf{p}})$ through the registered Merkle root r . If the verification succeeds, the stored information for each data entry becomes:

root: r , data: $\langle \mathbf{c} \rangle$, demo: $(\langle \tilde{\mathbf{c}} \rangle, \tilde{\mathbf{b}})$, tag: $(\epsilon, \$_{raw})$

- Upon receiving any foretaste request from the data consumer $\mathcal{P}_c$, each node responds with $(\langle \tilde{\mathbf{c}} \rangle, \tilde{\mathbf{b}})$. The consumer $\mathcal{P}_c$ then employs $\Pi_{ss}.rec$ to reconstruct the blinded demo $\tilde{\mathbf{c}}$, and calculates the demo dataset as $\tilde{\mathbf{m}} := \tilde{\mathbf{c}} - \tilde{\mathbf{b}}$. From the demo dataset $\tilde{\mathbf{m}}$ and data price $\$_{raw}$, the consumer evaluates whether the data quality is worth the price. If so, the consumer deposit $\$_{raw} - \$_\epsilon$ to the data market.
- Following the data consumer $\mathcal{P}_c$'s payment confirmation, each market node $\mathcal{P}_i$ informs the data owner $\mathcal{P}_o$. $\mathcal{P}_o$ proceeds to the next phase after receiving confirmations from at least $t + 1$ market nodes, which indicates the involvement of at least one honest node.

5.5 Distributed Data Perturbation

In this phase, the market nodes execute Π_{b-DNS} , and locally perturb the blinded data share with the generated Gaussian noise share.

- $\mathcal{P}_c$, $\mathcal{P}_o$, and the market nodes collectively initiate batched distributed noise sharing Π_{b-DNS} , to create (n, t) -Shamir's secret shares $\langle \xi \rangle$ of Gaussian random variables with batch size D . The noise vector ξ follows distribution $\mathcal{N}(\mu, \sigma^2)$, whose parameters are deterministically derived from the privacy budget ϵ .
- When each market node outputs $\langle \xi \rangle$, it calculates the noise-perturbed blinded data share $\langle \tilde{\mathbf{c}} \rangle$ from the locally retrieved blinded data share $\langle \mathbf{c} \rangle$ through equation: $\langle \tilde{\mathbf{c}} \rangle := \langle \mathbf{c} \rangle + \langle \xi \rangle$.
- Each node locally erases the blinded data share $\langle \mathbf{c} \rangle$ and stores $\langle \tilde{\mathbf{c}} \rangle$. Now, the stored information for each data entry includes:

root: r , perturbed data: $\langle \tilde{\mathbf{c}} \rangle$, demo: $(\langle \tilde{\mathbf{c}} \rangle, \tilde{\mathbf{b}})$, tag: $(\epsilon, \$_{raw})$

- Additionally, if Π_{b-DNS} fails, the market nodes initiate Π_{BFT} to reach a consensus, and refund to the data consumer.

5.6 Blinding Factor Establishment

In this phase, the blinding factor is confidentially and verifiably transmitted from the data owner to the data consumer. The security and utility analysis of the blinding factor is given in Appendix E.

- $\mathcal{P}_o$ transmits the blinding factor $\mathbf{b}$ to $\mathcal{P}_c$ through the secure and authenticated security channel. $\mathcal{P}_c$ then validates $\mathbf{b}$ by computing its Merkle root r' and broadcasting r' among the market nodes.
- Each market node locally checks whether r' matches the registered Merkle root r from the owner. If so, it inputs 1 into the Byzantine fault tolerance consensus Π_{BFT} to endorse this valid

Table 2: Accuracy with varying ϵ and γ

# Percentage (γ)	# Privacy Budget (ϵ)				
	$\epsilon \approx 1$	$\epsilon \approx 2$	$\epsilon \approx 4$	$\epsilon \approx 8$	$\epsilon \approx 16$
$\gamma = 3\%$	92.28	92.02	92.63	92.51	92.46
$\gamma = 15\%$	90.63	90.82	90.46	90.07	90.09
$\gamma = 31\%$	89.96	88.14	90.03	89.96	88.33
$\gamma = 63\%$	83.06	83.94	85.18	85.03	84.63
$\gamma = 73\%$	79.59	79.95	79.66	84.30	79.02
$\gamma = 83\%$	74.54	73.59	79.35	79.76	72.50
$\gamma = 93\%$	49.64	54.53	55.90	62.18	61.25
$\gamma = 100\%$	10.00	10.00	10.00	10.00	30.52
Non-private Baseline ($\gamma = 0\%, \epsilon = \infty$)					91.62

blinding factor establishment process. Otherwise, it inputs 0 to endorse the termination of the trading process.

- If all honest nodes agree on 0 through Π_{BFT} , the deposit is refund to the consumer. If Π_{BFT} outputs 1, the market nodes notify both $\mathcal{P}_o$ and $\mathcal{P}_c$, and they proceed to the next phase.

5.7 Bidirectional Fair Delivery

In this phase, the noise-perturbed data and the payment are delivered to the data consumer and the data owner, respectively.

- Each market node forwards the share of the noise-perturbed blinded data $\langle \tilde{c} \rangle$ with its index r to the consumer $\mathcal{P}_c$, and participates in another instance of Π_{BFT} with input 1.
- Once all market nodes agree on 1 that output from Π_{BFT} , they permit the transfer of the deposit to the data owner. Each node then sends $\langle \tilde{c} \rangle$ to the consumer if it hasn't sent before.
- When receiving enough shares $\langle \tilde{c} \rangle$ from different market nodes, the data consumer $\mathcal{P}_c$ invokes the reconstruction algorithm $\Pi_{\text{ss-rec}}$, from which the perturbed blinded data $\tilde{c}$ is interpolated.
- $\mathcal{P}_c$ calculates the noise-perturbed data $\tilde{m} := \tilde{c} - \mathbf{b}$ by subtracting the blinding factor $\mathbf{b}$ from the perturbed blinded data $\tilde{c}$.

Establishing the payment mechanism in a Byzantine-robust committee. For simplicity, the preceding discussion models the data market as a single logical payment entity. Next, we detail an instantiation for a Byzantine-resilient committee that accepts consumer deposits, issues refunds, and transfers deposits. In Xemis, all committee nodes jointly control an account on a payment platform using *threshold signature*: any $t + 1$ nodes can jointly produce a public-key-verifiable signature. After a consumer deposit, the account notifies all committee nodes. The nodes then decide, using BFT under the specified triggers, whether the trade succeeds or fails. The committee submits a threshold signature share to authorize the corresponding refund or payout, and the account executes upon collecting a valid signature. Xemis is compatible with many existing decentralized Web3 payment platforms, such as the blockchain system Ethereum. Moreover, the required threshold-signature-based account already exists in Web3 infrastructures, e.g., Lit Protocol [1].

5.8 Extension: Partial Data Perturbation

Focusing noise addition on sensitive data segments enhances the balance between data utility and privacy protection. The protocol presented above perturbs the raw dataset as a whole. Adapting it to target only sensitive data is straightforward. Specifically, the data owner additionally specifies a bit vector to the market nodes in data registration (Phase ①). The bit vector marks the sensitive parts in the raw data with 1. In distributed data sampling (Phase ②), the market nodes construct the demo dataset, while excluding the

marked sensitive data. In distributed data perturbation (Phase ③), noise injection is restricted to the specified sensitive segments. Consequently, the reconstructed data comprises both perturbed sensitive parts and unperturbed raw data portions. Evidenced by the evaluation in Section 6.1, partial data perturbation further enhances the utility of noise-perturbed data.

6 Evaluation

Testbed environment. Experiments were conducted on the machine with a 16-core Intel i9-12900KF CPU with 64GB of RAM and an Nvidia 4090 with 24GB of RAM. We utilize a Bash script to simulate a distributed environment consisting of n nodes. Each node is instantiated as an independent process bound to a unique network port, enabling inter-process message exchanges with a simulated WAN network of 400Mbps bandwidth and 4ms latency. The implementations are primarily developed in Python 3.7.13, using the 255-bit prime-order scalar field from BLS12-381, with the core cryptographic operations delegated to a Rust-based library from Zcash [49]. For fair comparison, we implement ODO [17], CSU19 [9], our b-DNS, and our Xemis using the same components, including the secret sharing [44]-based MPC [5], the reliable broadcast protocol from [14], and the BFT protocol from [12].

6.1 Utility Evaluation of Perturbed Data

We evaluate the utility of noise-perturbed data through the image classification task on CIFAR-10 with the ResNet-18 [21] network adopted. The task accuracy with respect to the ground truth labels is used as the utility matrix. We enhance image classification accuracy through data augmentation techniques along with model modifications through the training strategies in [38]. For hyperparameters, we set training epochs $e = 250$ and batch size $b = 128$.

Results. Table 2 reports classification accuracy across varying privacy budgets ϵ and noise augmentation ratios γ applied to the training set. Accuracy remains high for $\gamma < 31\%$ and declines as γ increases. Notably, adding noise to only 3% of the training data yields an accuracy that exceeds the non-private baseline, suggesting a robustness effect. Under a high-budget, low-noise configuration $\epsilon = 16$ on the full dataset, accuracy is 30.52%, whereas under a low-budget, high-noise configuration $\epsilon = 1$ on 63% of the data, accuracy is 83.06%. This indicates that partially perturbing sensitive data can offer better data utility and privacy protection. Appendix C provides additional results for perturbing the complete training set.

6.2 Evaluation of b-DNS

Experiment setup. This experiment benchmarks the performance of b-DNS by generating the shares of Gaussian samples for 3,072 Bytes of data. For parameters, the mean and variance of the generated Gaussian distribution is $\mu = 0$ and $\sigma^2 = 4$. The prototype is evaluated with market size $n = 4, 7, 16, 31, 64$, threshold $t = \lfloor n/3 \rfloor$ and distribution parameter $\kappa = 100, 300, 500, 700, 900$.

Results. We compare the runtime and bandwidth usage of our b-DNS with the MPC-based distributed noise sharing protocols ODO [17] and CSU19 [9]. Figure 3.(a) and 3.(b) present the runtime comparison with varying market size when the distribution parameter is fixed at $\kappa = 100$ and 900, respectively. Notably, b-DNS is 12.7–24.8× faster than ODO [17] and 19.1–37.2× faster than CSU19 [9]

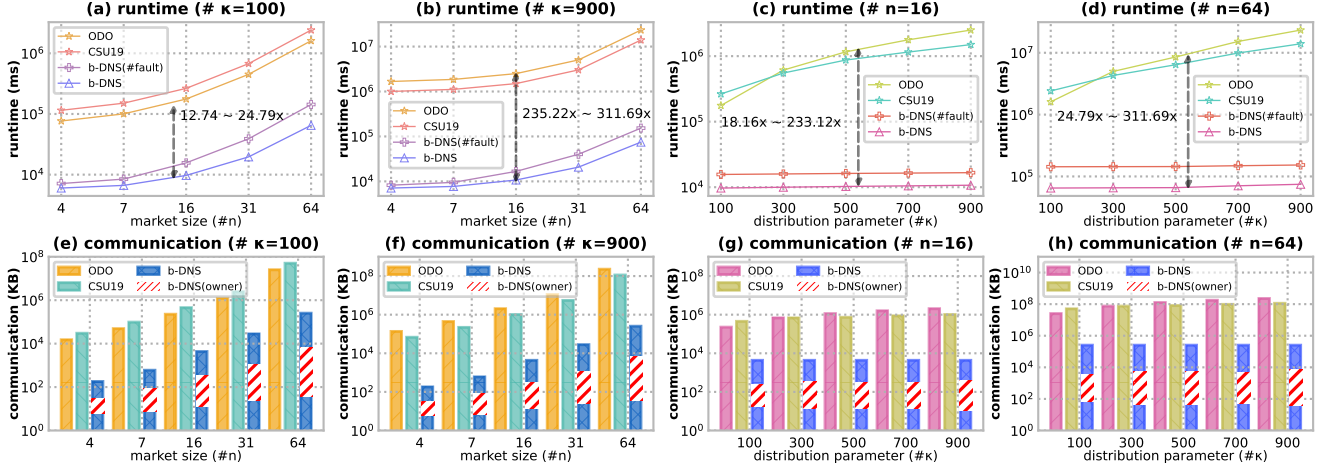
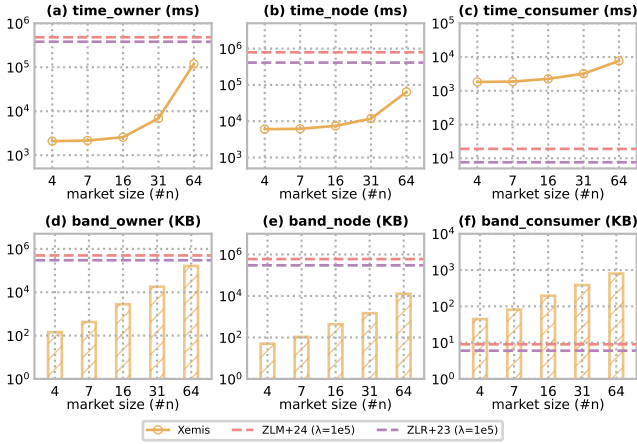
Figure 3: Evaluation of batched distributed noise sharing in runtime and bandwidth with varied n and κ 

Figure 4: Evaluation of Xemis and ZLM+24

when $\kappa = 100$, and their runtime reaches up to 311.7 $\times$ and 187.0 $\times$ than b-DNS when $\kappa = 900$. Moreover, when at most t nodes act maliciously, there is a slight 1.2–2.2 $\times$ increase in b-DNS’s runtime. As illustrated in Figure 3.(c) and (d), the growing distribution parameter hardly contributes to the runtime in b-DNS. Figure 3.(e), 3.(f), 3.(g) and 3.(h) illustrate the comparison of the transmitted message amount for all schemes. For a 16-node market, the owner’s bandwidth usage is only 1.3MB. The communications are at least 43.6 $\times$ in ODO [17] and 87.1 $\times$ in CSU19 [9] compared to b-DNS, and continuously grows with the distribution parameter.

6.3 Evaluation of Xemis

Experiment setup. This experiment benchmarks the performance of Xemis with data volume of 3,072 Bytes, distribution parameter $\kappa = 100$, market size $n = 4, 7, 16, 31, 64$, and threshold $t = \lfloor n/3 \rfloor$. **Baseline.** Xemis is the first DP-based data-trading system to offer controllable privacy with Byzantine robustness. For empirical comparison, we therefore benchmark against ZLM+24 [48] and ZLR+23 [49], two representative schemes that achieve controllable privacy under the semi-honest model. We implement it in ISO C++17 using the SEAL library at 128-bit security with a dimension of 8192 and negligible parameter $\lambda = 1e5$.

Results. Figure 4.(a), 4.(b) and 4.(c) present the runtime comparison with varying market size in Xemis, while Figure 4.(d), 4.(e), and 4.(f) depict their bandwidth comparisons. In Xemis, both runtime and bandwidth increase polynomially with the number of nodes. Although ZLM+24 [48] and ZLR+23 [49] are efficient at the consumer side, Xemis offers Byzantine robustness with much lower cost at the owner and market sides: its runtime is only 0.4–31.5% for the owner and 0.2–1.9% for the market, relative to ZLM+24 [48] and ZLR+23 [49]; Xemis provides more than 1.8 $\times$ higher bandwidth efficiency for the owner and more than 23.5 $\times$ for the market.

7 Conclusion

This paper presents Xemis, a decentralized privacy-preserving data trading scheme with controllable privacy and strong fairness under the Byzantine adversary. Leveraging our newly designed batched distributed noise sharing protocol, b-DNS, Xemis guarantees access minimization and controllable noise injection with low overhead. Moreover, Xemis designs a payment mechanism with sampling-based foretaste and BFT-based consensus that enables fair bidirectional delivery, fair value assessment, and fair privacy compensation simultaneously. We conduct theoretical analysis and practical evaluation on the performance of b-DNS and Xemis, as well as the utility of noise-perturbed data. The results demonstrate the efficiency, accuracy, and practicality of our proposed scheme.

8 Acknowledgments

This paper is supported by the National Key R&D Program of China (2025ZD0808500); the National Natural Science Foundation of China (T2425023, U21B2021, U22B2008, U21A20467, U2241213, 62202027, 62172025, 62472015); Beijing Natural Science Foundation (L251003); Zhejiang Provincial Natural Science Foundation of China (LMS25F020014); Lee Kong Chian Chair Professorship of Singapore Management University; Open Research Fund of the State Key Laboratory of Blockchain and Data Security, Zhejiang University; and the Fundamental Research Funds for the Central Universities, Beijing Advanced Innovation Center for Future Blockchain and Privacy Computing. W. Susilo is supported by the Australian Research Council (ARC) Laureate Fellowship (FL230100033).

References

- [1] Lit Association. [n. d.]. Lit Protocol. <https://www.litprotocol.com>.
- [2] AWS Data Exchange [n. d.]. <https://aws.amazon.com/data-exchange/>.
- [3] Akhil Bandarpalli, Xiaoyu Ji, Aniket Kate, Chen-Da Liu-Zhang, and Yifan Song. 2025. Computationally efficient asynchronous MPC with linear communication and low additive overhead. In *Annual International Cryptology Conference*. Springer, 261–294.
- [4] BDEX [n. d.]. <https://www.bdex.com/>.
- [5] Donald Beaver. 1992. Efficient multiparty protocols using circuit randomization. In *Advances in Cryptology—CRYPTO'91: Proceedings 11*. Springer, 420–432.
- [6] Michael Ben-Or, Ran Canetti, and Oded Goldreich. 1993. Asynchronous secure computation. In *Proceedings of the twenty-fifth annual ACM symposium on Theory of computing*, 52–61.
- [7] Adithya Bhat, Nibesh Shrestha, Zhongtang Luo, Aniket Kate, and Kartik Nayak. 2021. Randpipe—reconfiguration-friendly random beacons with quadratic communication. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 3502–3524.
- [8] Hui Cai, Yuanyuan Yang, Weiwei Fan, Fu Xiao, and Yanmin Zhu. 2023. Towards correlated data trading for high-dimensional private data. *IEEE Transactions on Parallel and Distributed Systems* 34, 3 (2023), 1047–1059.
- [9] Jeffrey Champion, Abhi Shelat, and Jonathan Ullman. 2019. Securely sampling biased coins with applications to differential privacy. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 603–614.
- [10] Kevin Choi, Aathira Manoj, and Joseph Bonneau. 2023. Sok: Distributed randomness beacons. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 75–92.
- [11] Ivan Damgård and Jesper Buus Nielsen. 2007. Scalable and unconditionally secure multiparty computation. In *Annual International Cryptology Conference*. Springer, 572–590.
- [12] Sourav Das, Sisi Duan, Shengqi Liu, Atsuki Momose, Ling Ren, and Victor Shoup. 2024. Asynchronous consensus without trusted setup or public-key cryptography. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 3242–3256.
- [13] Sourav Das, Vinith Krishnan, Irene Miriam Isaac, and Ling Ren. 2022. Spurt: Scalable distributed randomness beacon with transparent setup. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2502–2517.
- [14] Sourav Das, Zhuolun Xiang, and Ling Ren. 2021. Asynchronous data dissemination and its applications. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 2705–2721.
- [15] Datarade [n. d.]. <https://datarade.ai/>.
- [16] Sisi Duan, Xin Wang, and Haibin Zhang. 2023. FIN: practical signature-free asynchronous common subset in constant time. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 815–829.
- [17] Cynthia Dwork, Krishnamurthy Kenthapadi, Frank McSherry, Ilya Mironov, and Moni Naor. 2006. Our data, ourselves: Privacy via distributed noise generation. In *Advances in Cryptology—EUROCRYPT 2006: 24th Annual International Conference on the Theory and Applications of Cryptographic Techniques, St. Petersburg, Russia, May 28–June 1, 2006. Proceedings 25*. Springer, 486–503.
- [18] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. 2006. Calibrating noise to sensitivity in private data analysis. In *Theory of Cryptography: Third Theory of Cryptography Conference, TCC 2006, New York, NY, USA, March 4–7, 2006. Proceedings 3*. Springer, 265–284.
- [19] Cynthia Dwork, Aaron Roth, et al. 2014. The algorithmic foundations of differential privacy. *Foundations and Trends® in Theoretical Computer Science* 9, 3–4 (2014), 211–407.
- [20] Guojun Gao, Mingjun Xiao, Jie Wu, Sheng Zhang, Liusheng Huang, and Guiliang Xiao. 2019. DPDT: A differentially private crowd-sensed data trading mechanism. *IEEE Internet of Things journal* 7, 1 (2019), 751–762.
- [21] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.
- [22] Zaobo He and Zhipeng Cai. 2023. Trading Aggregate Statistics Over Private Internet of Things Data. *IEEE Trans. Comput.* (2023).
- [23] Feng Hu, Bing Chen, Jingyi Wang, Ming Li, Pan Li, and Miao Pan. 2019. MastDP: Matching based double auction mechanism for spectrum trading with differential privacy. In *2019 IEEE Global Communications Conference (GLOBECOM)*. IEEE, 1–6.
- [24] Aniket Kate, Gregory M Zaverucha, and Ian Goldberg. 2010. Constant-size commitments to polynomials and their applications. In *International conference on the theory and application of cryptography and information security*. Springer, 177–194.
- [25] Leslie Lamport, Robert Shostak, and Marshall Pease. 2019. The Byzantine generals problem. In *Concurrency: the works of Leslie Lamport*. 203–226.
- [26] Chuang Li, Aoli He, Yanhua Wen, Gang Liu, and Anthony Theodore Chronopoulos. 2023. Optimal trading mechanism based on differential privacy protection and stackelberg game in Big Data market. *IEEE Transactions on Services Computing* 16, 5 (2023), 3550–3563.
- [27] Chao Li, Daniel Yang Li, Gerome Miklau, and Dan Suciu. 2014. A theory of pricing private data. *ACM Transactions on Database Systems (TODS)* 39, 4 (2014), 1–28.
- [28] Jinguo Li, Yun Ni, Jin Zhang, Jie Yu, and Yin He. 2024. A Contract-Based Privacy-Preserving Longitudinal Data Trading Mechanism for IoT. *IEEE Internet of Things Journal* (2024).
- [29] Jinfei Liu, Jian Lou, Junxu Liu, Li Xiong, Jian Pei, and Jimeng Sun. 2021. Dealer: An end-to-end model marketplace with differential privacy. *Proceedings of the VLDB Endowment* 14, 6 (2021).
- [30] Zewei Liu, Chunqiang Hu, Hui Xia, Tao Xiang, Baolin Wang, and Jiajun Chen. 2022. SPDTS: a differential privacy-based blockchain scheme for secure power data trading. *IEEE Transactions on Network and Service Management* 19, 4 (2022), 5196–5207.
- [31] Yuan Lu, Zhenliang Lu, Qiang Tang, and Guiling Wang. 2020. Dumbo-mvba: Optimal multi-valued validated asynchronous byzantine agreement, revisited. In *Proceedings of the 39th symposium on principles of distributed computing*. 129–138.
- [32] MetaMask. [n. d.]. Web3Auth. <https://web3auth.io/docs/infrastructure/nodes-and-dkg>.
- [33] Achour Mostéfaoui, Hamouma Moumen, and Michel Raynal. 2014. Signature-free asynchronous Byzantine consensus with $t < n/3$ and $O(n^2)$ messages. In *Proceedings of the 2014 ACM symposium on Principles of distributed computing*. 2–9.
- [34] Satoshi Nakamoto. 2008. Bitcoin whitepaper. URL: <https://bitcoin.org/bitcoin.pdf> (17.07.2019) 9 (2008), 15.
- [35] Chaoyue Niu, Zhenzhe Zheng, Fan Wu, Shaojie Tang, Xiaofeng Gao, and Guihai Chen. 2019. ERATO: Trading noisy aggregate statistics over private correlated data. *IEEE Transactions on Knowledge and Data Engineering* 33, 3 (2019), 975–990.
- [36] Jian Pei. 2020. A survey on data pricing: from economics to data science. *IEEE Transactions on Knowledge and Data Engineering* 34, 10 (2020), 4586–4608.
- [37] Michael O Rabin. 1983. Randomized byzantine generals. In *24th annual symposium on foundations of computer science (sfcs 1983)*. IEEE, 403–409.
- [38] ResNet18Cifar10 [n. d.]. https://github.com/ZOMIN28/ResNet18_Cifar10. 95.46.
- [39] Victor Shoup and Nigel P Smart. 2024. Lightweight asynchronous verifiable secret sharing with optimal resilience. *Journal of Cryptology* 37, 3 (2024), 27.
- [40] the U.S. Healthcare Data dataset [n. d.]. <https://datarade.ai/data-products/u-s-healthcare-data-national-practitioner-level-ratespot>.
- [41] Sri Aravinda Krishnan Thyagarajan, Pratik Soni, and Ke Wu. 2024. Game-Theoretically Fair Distributed Sampling. In *Annual International Cryptology Conference*. Springer, 207–239.
- [42] Victor Van Der Veen, Yanick Fratantonio, Martina Lindorfer, Daniel Gruss, Clémentine Maurice, Giovanni Vigna, Herbert Bos, Kaveh Razavi, and Cristiano Giuffrida. 2016. Drammer: Deterministic rowhammer attacks on mobile platforms. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*. 1675–1689.
- [43] Shengling Wang, Lina Shi, Qin Hu, Junshan Zhang, Xiuzhen Cheng, and Jiguo Yu. 2021. Privacy-aware data trading. *IEEE Transactions on Information Forensics and Security* 16 (2021), 3916–3927.
- [44] Xinxin Xing, Yizhong Liu, Boyang Liao, Jianwei Liu, Bin Hu, Xun Lin, Yuan Lu, and Tianwei Zhang. 2025. GoSSamer: Lightweight and Linear-Communication Asynchronous (Dynamic Proactive) Secret Sharing and the Applications. *Cryptology ePrint Archive* (2025).
- [45] Shuming Xiong, Pengchao Chen, Shusheng Ge, and Qiang Ni. 2024. SFOM-DT: A Secure and Fair One-to-Many Data Trading Scheme Based on Blockchain. *IEEE Transactions on Information Forensics and Security* (2024).
- [46] Liang Xue, Jianbing Ni, Dongxiao Liu, Xiaodong Lin, and Xuemin Shen. 2023. Blockchain-based fair and fine-grained data trading with privacy preservation. *IEEE Trans. Comput.* 72, 9 (2023), 2440–2453.
- [47] Fan Yang, Xiaofeng Liao, Xinyu Lei, Nankun Mu, and Di Zhang. 2023. Towards privacy-preserving and practical data trading for aggregate statistic. *IEEE Transactions on Sustainable Computing* 9, 3 (2023), 452–463.
- [48] Man Zhang, Xinghua Li, Yinbin Miao, Bin Luo, Wanyun Xu, Yanbing Ren, and Robert H Deng. 2024. Privacy-Preserved Data Disturbance and Truthfulness Verification for Data Trading. *IEEE Transactions on Information Forensics and Security* (2024).
- [49] Man Zhang, Xinghua Li, Yanbing Ren, Bin Luo, Yinbin Miao, Ximeng Liu, and Robert H Deng. 2023. Privacy-preserved data trading via verifiable data disturbance. *IEEE Transactions on Dependable and Secure Computing* (2023).

A Security Analysis of b-DNS

A.1 Definition

Definition 1 ((Batched) distributed noise sharing). An (n, t, D, ϵ) - Π_{b-DNS} protocol distributively generates the Shamir’s secret shares of a vector of D Gaussian noise ξ among n participating nodes with a privacy budget of ϵ . The protocol is *batched* if $D > 1$. Π_{b-DNS}

satisfies the following properties if at most $t = \lfloor 1/3n \rfloor$ nodes are corrupted by an Byzantine adversary:

- **Correctness:** If all honest nodes participate in the protocol, then each honest node outputs an (n, t) -threshold share $\langle \xi \rangle$ of noise ξ with batch size D , such that any subset of more than t valid shares can be reconstructed into ξ through $\Pi_{ss}.rec$. The noise vector ξ is generated via a centered-binomial approximation to a Gaussian with distribution parameter κ .
- **Completeness:** If an honest node output its share $\langle \xi \rangle$, then all honest nodes will eventually outputs its share.
- **Secrecy:** The adversary's advantage in guessing ξ is negligible.

A.2 Rational Unbiased Seeds Generation

Players and strategies. There are two rational players: a data owner $\mathcal{P}_o$ and a data consumer $\mathcal{P}_c$. Each independently selects a distribution to generate a single bit:

$$B_o \sim \text{bern}(p), \quad B_c \sim \text{bern}(q)$$

where the strategies are $p, q \in [0, 1]$, B_o is sampled by the owner, and B_c is sampled by the consumer. The owner first secret-shares B_o via $\Pi_{ss}.share$; only after this sharing succeeds does the consumer broadcast B_c . Thus, the two parties sample simultaneously without observing each other's sample. The data market performs XOR on the sharing $\langle B_o \rangle$ with the bits B_c , resulting in $\langle B \rangle := \langle B_o \oplus B_c \rangle$, where $B \in \{0, 1\}$ (Step ②, b-DNS). There is a pool of fair bits $U_1, \dots, U_K \sim \text{bern}(1/2)$, independent of B . Define

$$\langle V_i \rangle := \langle B \oplus U_i \rangle, \quad \langle S \rangle := \sum_{i=1}^K \langle V_i \rangle$$

We center and scale $\langle S \rangle$ to obtain the sharing of an approximately discrete Gaussian (Step ④, b-DNS). The owner $\mathcal{P}_o$ prefers more noise, while the consumer $\mathcal{P}_c$ prefers less noise, which we abstract as $B = 1$ and $B = 0$, respectively. The expected utility for $\mathcal{P}_o$ is:

$$\tilde{u}_o(p, q) := \mathbb{P}[B = 1] = p(1 - q) + (1 - p)q = p + q - 2pq.$$

The expected utility for $\mathcal{P}_c$ is:

$$\tilde{u}_c(p, q) := \mathbb{P}[B = 0] = 1 - \tilde{u}_o(p, q).$$

Equivalently, to make the game strictly zero-sum:

$$u_o(p, q) := \tilde{u}_o(p, q) - \frac{1}{2} = p(1 - 2q) + q - \frac{1}{2}, \quad u_c(p, q) := -u_o(p, q).$$

Basic probability lemma and its corollary. For a random bit $X \in \{0, 1\}$, define its bias as $\mathbb{B}(X) := \mathbb{P}[X = 1] - \frac{1}{2}$.

Lemma 1 (XOR bias formula). If $B_o \perp B_c$, with $B_o \sim \text{bern}(p)$ and $B_c \sim \text{bern}(q)$, then

$$\begin{aligned} \mathbb{B}(B_o \oplus B_c) &= \mathbb{P}[B = 1] - \frac{1}{2} = p(1 - q) + (1 - p)q - \frac{1}{2} \\ &= -2(p - \frac{1}{2})(q - \frac{1}{2}) \end{aligned}$$

Corollary 1. (1) If one side is unbiased ($p = \frac{1}{2}$ or $q = \frac{1}{2}$), then B is unbiased. (2) If $|p - \frac{1}{2}| \leq \epsilon$ and $|q - \frac{1}{2}| \leq \epsilon$, then $|\mathbb{B}(B_o \oplus B_c)| \leq 2\epsilon^2$

The unique Nash equilibrium. Next, we prove that the unique mixed Nash equilibrium is unbiased.

Theorem 1 (Best responses and equilibrium). In the above zero-sum game, the unique Nash equilibrium is $p^* = q^* = \frac{1}{2}$. At this equilibrium $B \sim \text{bern}(1/2)$ is unbiased.

PROOF. By definition $u_o(p, q) = \mathbb{P}[B = 1] = p(1 - 2q) + q$, which is linear in p . Fix q , the best response is:

$$q < \frac{1}{2} \Rightarrow p = 1; \quad q > \frac{1}{2} \Rightarrow p = 0; \quad q = \frac{1}{2} \Rightarrow p \text{ arbitrary.}$$

Symmetrically (minimizing u_o), fixing p :

$$p < \frac{1}{2} \Rightarrow q = 0; \quad p > \frac{1}{2} \Rightarrow q = 1; \quad p = \frac{1}{2} \Rightarrow q \text{ arbitrary.}$$

The best response correspondences intersect only at $p = q = \frac{1}{2}$, which is thus the unique mixed Nash equilibrium. By Lemma 1, B is unbiased at this point. $\square$

This is the continuous version of the classic matching-pennies game; the same conclusion follows from the minimax theorem. Moreover, in zero-sum games, any (coarse) correlated equilibrium has marginal strategies equal to the minimax strategies, hence even with a public correlating device the marginals remain $p = q = \frac{1}{2}$.

Approximate equilibrium and robustness. Define a strategy profile (p, q) as an ϵ -approximate Nash equilibrium if no unilateral deviation can improve a player's payoff by more than ϵ .

Proposition 1 (Approximate unbiasedness). (1) If at least one side plays the equilibrium exactly ($p = \frac{1}{2}$ or $q = \frac{1}{2}$), then B is unbiased. (2) In this zero-sum game, if (p, q) is an ϵ -approximate Nash equilibrium, then $|\mathbb{B}(B_o \oplus B_c)| \leq 2\epsilon^2$. A direct proof compares the current payoff to the best pure response payoff: for any p, q ,

$$\begin{aligned} \max_{p \in [0, 1]} u_o(p, q) &= \max\{q, 1 - q\} = \frac{1}{2} + |q - \frac{1}{2}|, \\ \min_{q \in [0, 1]} u_o(p, q) &= \min\{p, 1 - p\} = \frac{1}{2} - |p - \frac{1}{2}|. \end{aligned}$$

An ϵ -approximate Nash equilibrium requires:

$$\frac{1}{2} - |p - \frac{1}{2}| \geq \frac{1}{2} - \epsilon, \quad \frac{1}{2} + |q - \frac{1}{2}| \leq \frac{1}{2} + \epsilon,$$

which yields $|p - \frac{1}{2}|, |q - \frac{1}{2}| \leq \epsilon$. Plugging into Lemma 1 gives the quadratic bound.

A.3 Security Analysis

Lemma 2 (Berry-Esseen Theorem). Let $X_1, \dots, X_K$ be independent and identically distributed (i.i.d.) random variables with $\mathbb{E}[X] = \mu_b$, $\mathbb{E}[|X - \mu_b|^2] = \sigma_b^2 > 0$. Let $S_K = \sum_{i=1}^K X_i$ denote their sum. The standardized version of S_K is given by $Z_K = (S_K - \kappa\mu_b)/(\sqrt{\kappa}\sigma_b)$. There exists a constant C_b such that:

$$\sup |\Psi_{Z_K}(x) - \Phi(x)| \leq \frac{C_b}{\sigma_b^3 \sqrt{\kappa}} \mathbb{E}[|X - \mu_b|^3]$$

$\Psi_{Z_K}(x)$ is the cumulative distribution function (CDF) of the standardized sum Z_K , and $\Phi(x)$ is the CDF of the standard Gaussian.

Lemma 3 (Correctness of Π_{b-DNS}). The protocol Π_{b-DNS} in Section 4 satisfies the correctness of distributed noise sharing.

PROOF. In Step ①, the correctness of $\Pi_{ss}.share$ guarantees that if the data owner is rational and behaves as in Theorem 1, all honest nodes will participate in the protocol and outputs a valid share $\langle s \rangle$ of the bit vector s even under the Byzantine adversary. The subsequent MPC verification further guarantees $s \in \{0, 1\}$ with Byzantine resilience. In Step ②, on assuming that the data consumer is also rational and behaves as in Theorem 1, it reliably broadcasts bit vector s' among market nodes. The correctness of reliable broadcast guarantees that all nodes receive the same vector s' . Thus, all

honest market nodes perform XOR operation for $\langle \text{seed} \rangle = \langle s \oplus s' \rangle$. According to Theorem 1, the unique Nash equilibrium leads to a uniform and random distribution of seed over $\{0, 1\}$. In Step ③, all honest nodes locally retrieve the offline-generated coin pool coin of size $D \times \kappa$, extract each of its column, perform entry-wise XOR on $\langle \text{seed} \rangle$ and each column, thus obtaining the matrix $\langle \mathbf{u} \rangle$ of size $D \times \kappa$, which consists of the shares of uniformly and randomly distributed bits. After summing all the κ columns in $\langle \mathbf{u} \rangle$, all nodes obtain $\langle \xi \rangle$ of size D . Guaranteed by the additive homomorphic property of $\Pi_{\text{ss.share}}$, all honest nodes will eventually output $\langle \xi \rangle$, such that any subset of more than t shares can be reconstructed into ξ through $\Pi_{\text{ss.rec}}$. By Lemma 2, denote $\mathbf{Z}_\kappa$ as the standardized version of ξ , there is $\sigma_b = 1/2$ and $\mathbb{E}[|X - \mu_b|^3] = 1/8$, then:

$$\sup |\Psi_{\mathbf{Z}_\kappa}(x) - \Phi(x)| \leq \frac{C_b}{\sqrt{\kappa}}$$

In the Berry-Esseen theorem, the absolute constant C_b is commonly taken as 0.56 for a convenient universal bound. On assuming that τ is a negligible security parameter, when $\kappa \geq 0.3136/\tau^2$, there is $\sup |\Psi_{\mathbf{Z}_\kappa}(x) - \Phi(x)| \leq \tau$. Thus, the vector of noise ξ output in Step ④ is sampled from designated Gaussian distribution $\mathcal{N}(\mu^*, \sigma^*)$ with distribution parameter κ . $\square$

Lemma 4 (Completeness of $\Pi_{\text{b-DNS}}$). The protocol $\Pi_{\text{b-DNS}}$ in Section 4 satisfies the completeness of distributed noise sharing.

PROOF. The completeness of $\Pi_{\text{ss.share}}$ [44] ensures that if an honest node outputs the share $\langle s \rangle$ in Step ①, then every honest node eventually outputs its share, even in the presence of a Byzantine adversary. The completeness of the reliable broadcast protocol [14] likewise ensures that if an honest node outputs the vector s' in Step ②, then every honest node outputs s' under a Byzantine adversary. By analogy with Lemma 3, the completeness of $\Pi_{\text{b-DNS}}$ follows directly. $\square$

Lemma 5 (Secrecy of $\Pi_{\text{b-DNS}}$). The protocol $\Pi_{\text{b-DNS}}$ in Section 4 satisfies the secrecy of distributed noise sharing.

PROOF. The secrecy property of $\Pi_{\text{ss.share}}$ guarantees that the secret sharing process reveals no information about the shared bits s of the data owner, if the Byzantine adversary corrupts up to $t < 1/3$ nodes. After deriving the Gaussian shares $\langle \xi \rangle$, it further ensures that the adversary cannot observe any information about ξ from up to t shares. Moreover, the MPC-based verification also reveals no information about s except for what is given by $s \in \{0, 1\}$. As each entry in $\langle \xi \rangle$ is flipped from each entry in $\langle s \rangle$, each value in ξ has two possible outcomes from the perspective of $\mathcal{A}$, resulting in 2^D possible values for ξ . Thus, the adversaries' advantage $\text{Adv}_{\mathcal{A}}$ in successfully inferring ξ is 2^{-D} , which is negligible for large enough batch size D . Hence, the lemma is proven. $\square$

Theorem 2. The protocol $\Pi_{\text{b-DNS}}$ in Section 4 is a batched distributed noise sharing protocol that satisfies correctness, completeness, and secrecy properties as in Definition 1, against any P.P.T Byzantine adversary corrupting up to $t < 1/3$ nodes, on assuming that the data owner and data consumer are rational entities.

PROOF. The proof of this theorem follows directly from Lemma 3, Lemma 4, and Lemma 5. $\square$

B Security Analysis of Xemis

B.1 Security Analysis

Lemma 6. Assume that the blinded data provided by the owner $\mathcal{P}_o$ is $c = m + b$, the registered blinding factor is b^* , and the blinding factor sent to the consumer $\mathcal{P}_c$ is b' . If the $\mathcal{P}_c$ obtains the perturbed and blinded data $\tilde{c}$, then there is $b = b^* = b'$.

PROOF. If the consumer outputs the data $\tilde{c}$ from $\Pi_{\text{ss.rec}}$, then all honest nodes must have agreed on 1 through Π_{BFT} in Phase ④, blinding factor establishment. This confirms $r = r'$, where r and r' are the Merkle roots of the registered blinding factor b^* and the transmitted blinding factor b' , respectively. According to the collusion-resistant property of the hash function, there is $b^* = b'$.

Additionally, if the consumer outputs the data $\tilde{c}$ from $\Pi_{\text{ss.rec}}$, then it must have obtained the sampled data $\tilde{m} = (\tilde{m} + \tilde{b}) - \tilde{b}'$ and paid in the Phase ②, distributed data sampling. The market nodes collectively guarantee that $\tilde{b}' = b^*$ by verifying the Merkle proof. The consumer then verifies the legitimacy of the sample data before payment, and if it pays, there is $\tilde{b} = \tilde{b}'$. We will prove that under these circumstances, the probability that $b \neq b'$ is negligible. Assume that the sampling ratio is θ_1 , the proportion of unequal entries in b and b' is θ_2 , and the data scale is D . The probability that all sampled blinding factors are equal is:

$$\mathbb{P}[\tilde{b} = \tilde{b}'] = \frac{C((1 - \theta_1)D, \theta_2 D)}{C(D, \theta_2 D)}$$

After applying Stirling's formula, there is:

$$\mathbb{P}[\tilde{b} = \tilde{b}'] \approx \left(\frac{(1 - \theta_1)^{1 - \theta_1} (1 - \theta_2)^{1 - \theta_2}}{(1 - \theta_1 - \theta_2)^{1 - \theta_1 - \theta_2}} \right)^D$$

Since $\theta_1 \neq 0$, and if $\theta_2 \neq 0$, then for large enough D , the above probability is negligible. Thus, the proportion of unequal entries in b and b' is $\theta_2 = 0$, then there is $b = b'$. This concludes the proof. $\square$

Theorem 3. Xemis achieves (1) controllable privacy, including access minimization and controllable noise injection; (2) strong fairness, including fair bidirectional delivery, fair value assessment, and fair privacy compensation, as long as the data owner and data consumer are rational entities, and up to $t < 1/3n$ market nodes are corrupted by the Byzantine adversary.

PROOF. Next, we prove that the privacy-preserving data trading protocol, Xemis, satisfies controllable privacy and strong fairness under the Byzantine adversary.

Controllable privacy.

First, we prove that Xemis satisfies access minimization. According to the protocol, the view of the consumer $\mathcal{P}_c$ consists of (1) the perturbed blinded data $\tilde{c}$, (2) the blinding factor b and (3) the demo dataset $\tilde{m}$; while the view of each market node consists of (1) the registered information related to the data, (2) the share of blinded data $\langle c \rangle$, (3) the share of the noise $\langle \xi \rangle$ and (4) the share of demo dataset $\langle \tilde{m} \rangle$. As long as the malicious market nodes are less than $1/3n$, the secrecy of $\Pi_{\text{ss.share}}$ guarantees that only the data owner $\mathcal{P}_o$ and the data consumer $\mathcal{P}_c$ can observe the perturbed blinded data c . As the blinding factor b is transmitted in a secure channel, it can be proved that the noise-perturbed data $\tilde{m}$ is available solely to $\mathcal{P}_o$ and the paying $\mathcal{P}_c$; all other entities, including the data market,

learn nothing about it unless the owner or the consumer explicitly discloses it. Moreover, by Lemma 5, the advantage of the consumer in guessing the raw data $\mathbf{m}$ from the perturbed data $\tilde{\mathbf{m}}$ is 2^{-D} , which is negligible given the data scale D is large enough. This further ensures that the raw data $\mathbf{m}$ is only accessible to the data owner.

Next, we prove that Xemis satisfies controllable noise injection. The correctness of the protocol Π_{b-DNS} guarantees that each node outputs a share $\langle \xi \rangle$ of Gaussian noise ξ under the per-negotiated privacy budget ϵ with Byzantine resilience. Each honest node will calculate the perturbed blinded data share $\langle \tilde{c} \rangle$ by adding the noise share $\langle \xi \rangle$ to the blinded data share $\langle c \rangle$. If the consumer $\mathcal{P}_c$ obtains the perturbed data $\tilde{\mathbf{m}}$, then $\tilde{\mathbf{m}} := \tilde{\mathbf{c}} - \mathbf{b}$, where $\tilde{\mathbf{c}}$ is reconstructed from the shares sent from market nodes. Guaranteed by $\Pi_{ss.share}$ and $\Pi_{ss.rec}$, if the Byzantine adversary corrupts up to t nodes, then there must be $\tilde{\mathbf{c}} = \mathbf{c} + \xi$. According to Lemma 6, there is $\mathbf{c} = \mathbf{m} + \mathbf{b}$. Thus, it can be proved that $\tilde{\mathbf{m}} = \mathbf{m} + \xi$, where ξ is the noise vector generated from the pre-negotiated privacy budget.

Strong fairness.

First, we prove that Xemis satisfies the fair bidirectional delivery property. If the consumer $\mathcal{P}_c$ receives the noise-perturbed data, then at least $t + 1$ honest nodes must have sent the perturbed blinded data share $\langle \tilde{c} \rangle$ in Phase (5), bidirectional fair delivery. This further indicates that the first consensus instance Π_{BFT} in Phase (4) outputs 1. Thus, all honest nodes will eventually send the share $\langle \tilde{c} \rangle$ to the consumer $\mathcal{P}_c$. These honest nodes then invoke the second consensus instance Π_{BFT} in Phase (5) to reach a consensus and transfer the corresponding deposit to the data owner $\mathcal{P}_o$. Additionally, if $\mathcal{P}_o$ receives the payment, then the second consensus instance Π_{BFT} must have output 1 in Phase (5). This indicates that all honest nodes will send the perturbed and blinded data share $\langle \tilde{c} \rangle$ to the consumer, from which the perturbed blinded data $\tilde{\mathbf{c}} = \tilde{\mathbf{m}} + \mathbf{b}$ is reconstructed through $\Pi_{ss.rec}$. Moreover, if an honest market node sends its share, then the first consensus instance in Phase (4) must have output 1. This happens only after the consumer has paid and received the valid blinding factor $\mathbf{b}$ in Phase (3) and Phase (4). Thus, the consumer can locally calculate the noise-perturbed data $\tilde{\mathbf{m}}$.

Next, we prove that Xemis satisfies the fair value assessment property. The market nodes distributively sample some random indices according to the sample percentage as specified by the data owner, and assemble the corresponding blinded data share into the blinded demo share, from which the consumer can reconstruct the demo dataset and evaluate whether the raw data matches its price.

Assuming a pricing mechanism that sets the price of noise-perturbed data to $\$_{data} := \$_{raw} - \$_{\epsilon}$, where $\$_{raw}$ is the raw data price, and $\$_{\epsilon}$ is the privacy budget deterministically calculated from the privacy budget ϵ , Xemis satisfies fair privacy compensation. $\square$

C Utility Evaluation on Complete Dataset

Table 3 reports the image-classification accuracy with noise injected into all training samples under varying privacy budgets ϵ at 8, 16, 24, 32, 48, 64, 128, 256. As the privacy budget decreases, model accuracy declines. Specifically, for $\epsilon \geq 256$, the noise has a negligible impact on accuracy; for $\epsilon \geq 48$, the accuracy remains high; and at $\epsilon = 8$, the model completely fails on the ten-class classification

task, indicating that the features are effectively erased by the noise. In addition, models trained on data perturbed with Xemis and with standard Gaussian noise achieve nearly the same accuracy, with the latter slightly higher. This suggests that at $\kappa = 100$, Xemis noise may introduce a slightly stronger perturbation.

Table 3: Accuracy with varying privacy budget ($\gamma = 100\%$)

Method		$\epsilon = 256$	$\epsilon = 128$	$\epsilon = 64$	$\epsilon = 48$	$\epsilon = 32$	$\epsilon = 24$	$\epsilon = 16$	$\epsilon = 8$
Gaussian	Xemis [†]	91.69	89.51	84.81	80.92	67.81	52.38	27.98	10.00
	Standard	90.94	90.3	84.65	81.78	69.48	47.83	30.52	10.00
Non-private Baseline ($\gamma = 0\%$, $\epsilon = \infty$)								91.62	

[†] The distribution parameter in Xemis is set as $\kappa = 100$.

D Elaboration on Asynchronous Network

In this work, we consider a practical asynchronous network, which operates without any synchronous or partial-synchronous timing assumptions, and is immune to any failures under synchrony loss caused by the Byzantine adversary. In an asynchronous network, the adversary can arbitrarily delay and reorder the messages on the p2p channels. However, messages transmitted between nodes are guaranteed to be eventually delivered. Accordingly, all the building blocks we utilized are instantiated using asynchronous protocols.

E Security and Utility of the Blinding Factor

Security: As discussed in Section 3.4, the data owner and consumer communicate over a secure and authenticated channel, so an external adversary cannot observe the blinding factor from the channel. Consequently, the factor is learnable only if it is explicitly disclosed by the owner and the consumer, which is equivalent to revealing the raw data. Such data disclosure is against the trading parties' economic incentives and is typically constrained by contractual and legal obligations. Accordingly, we assume the probability of such disclosure is negligible in this work.

Utility: In Xemis, the blinding factor provides further guarantees beyond the assumption that an adversary can corrupt up to 1/3 of the market nodes: as long as both $\mathcal{P}_o$ and $\mathcal{P}_c$ do not reveal it, the market cannot recover the noise-perturbed data, even if all market nodes are Byzantine corrupted.

F Establishing mutually distrustful nodes

In this appendix, we provide a well-established Web3 solution for initializing mutually distrustful nodes. In deployed Web3 systems, mutual distrust is typically enforced by the permissioned committee composed of large, independently governed organizations with public identities and compliance exposure. For example, the Torus Network [32] in MetaMask runs a mainnet with about ten independent operators, such as Binance and Tendermint. Cross-organization collusion entails reputational and legal risk. Committee membership is curated, auditable, revocable, and economically incentivized. Deploying Xemis on such committees makes the assumption $t < n/3$ plausible and achieves higher efficiency than permissionless networks like Bitcoin [34]. Moreover, this design excludes data owners and consumers from serving as market nodes, eliminating their potential misbehavior and collusion in that role.